A Method for Composing Concerns into a Unified Domain Model in Domain-Driven Design

Van-Vinh Le[1,2], Nhat-Hoang Nguyen[1],
Duc-Quyen Nguyen[1], and Duc-Hanh Dang[1]*

[1] Faculty of Information Technology,
VNU University of Engineering and Technology, Hanoi, Vietnam
[2] Vinh University of Technology Education
Email: {21028005|24025170|21020388|hanhdd}@vnu.edu.vn

Abstract. Domain-Driven Design (DDD) emphasizes iterative development around a rich domain model to align developers and domain experts. While ubiquitous language and Domain-Specific Languages (DSLs) improve expressiveness and maintainability, modern systems often require multiple heterogeneous DSLs to cover diverse concerns. Existing DDD approaches, however, lack systematic methods to compose such DSLs, resulting in fragmented models and limited automation. Although meta-modeling offers a standard way to define DSLs, it is often rigid and framework-dependent. This paper introduces a novel method for composing heterogeneous concern-specific DSLs into a unified domain model within DDD. Each DSL is defined with a well-defined syntax and formal semantics, and is integrated via an annotation-based composition mechanism at the abstract syntax tree level. This ensures concern orthogonality, model cohesion, and supports consistency checking, automated code generation, and traceability. The approach is implemented using MPS and the `JDA` framework, and validated through representative case studies, advancing modular and executable domain modeling for complex systems.

Keywords: DDD · Domain Model · Domain Concern · AST · DSL

1 Introduction

The increasing complexity, scale, and heterogeneity of modern software systems have intensified the need for methodologies that effectively bridge domain knowledge and technical implementation. Domain-Driven Design (DDD) has emerged as a prominent approach to this challenge, advocating iterative development around a semantically rich domain model that encapsulates the core logic and rules of the problem domain [7, 12]. Central to DDD is the consistent use of a Ubiquitous Language (UL), which fosters effective communication and shared understanding between domain experts and developers throughout the software lifecycle. To operationalize the UL and align it with implementation artifacts, Domain-Specific Languages (DSLs) have been widely adopted as expressive means of encoding domain concepts directly in source code [9]. DSLs

* Corresponding author

tightly couple design models with implementation, thereby improving software correctness, maintainability, and extensibility [4, 8, 18, 19].

Defining and integrating multiple DSLs—each a *concern-specific DSL* representing a particular concern such as business logic, security, user interfaces, and performance—is essential for capturing the diverse requirements of complex software systems [10, 11].

Current approaches often address concerns in isolation, leading to tight coupling and limited extensibility. Recent efforts [15] have focused on defining metamodels based on UML metaconcepts to support the composition and unification of concern-specific DSLs. Within the method, the abstract syntax is defined through meta-concepts in an external DSL, while a one-to-one mapping to an annotation-based DSL provides an internal DSL representation within the host language. Nevertheless, significant limitations remain: integration processes are typically semi-automated and error-prone, rely heavily on high-quality mappings between DSLs, and offer limited support for complex or cross-cutting concerns. Scalability also becomes problematic as the number and complexity of concerns increase, while existing approaches still fail to capture sophisticated structural and behavioral aspects [5, 14].

In this paper, we propose a methodology for composing heterogeneous concern-specific DSLs into a unified domain model. The basic idea is to adopt an Abstract Syntax Tree (AST)-based method, which emphasizes simplicity, direct manipulation without parsing, support for multiple syntactic views, and provides a solid basis for consistency checking, executable code generation, and end-to-end traceability. Each DSL is uniformly defined in terms of abstract syntax (AS), concrete syntax (CS), and semantics, establishing a coherent foundation for cross-concern integration. Unification is achieved through an annotation-based composition mechanism at the AST level, which interrelates DSLs and binds their semantics to core domain elements. This ensures the orthogonality of concern-specific DSLs and preserves the cohesion of the unified domain model. We implement and validate the approach using the projectional editing capabilities of JetBrains MPS [3] and the DDD framework `JDA` [6], demonstrating its feasibility and effectiveness through real-world case studies.

In brief, the contributions of this paper are as follows:

- A novel AST-based methodology for composing heterogeneous concern-specific DSLs into the unified domain model, referred to as UDML.
- A complete definition of the syntax and semantics of UDML, establishing a coherent modeling foundation for cross-concern integration.
- A tool-supported implementation on the JetBrains MPS platform, advancing the state of the art in modular and extensible domain modeling.

The rest of the paper is organized as follows: Section 2 motivates our work with examples and background. Section 3 explains the basic idea of our approach. Section 4 provides the syntax and semantics for UDML and the mechanism to compose concerns. Tool support and experiments are explained in Section 5, which discusses the proposed language. Section 6 surveys related work. Finally, Section 7 concludes the paper with a discussion of future work.

2 Motivating Example and Background

This section motivates our work through an example and provides the necessary background.

2.1 Motivating Example

We use the `CourseMan` domain, adapted from [14], as a motivating example. Fig. 1 presents the `CourseMan` domain model, based on OMG/UML, combining a class diagram with concern-specific DSLs. It includes three main classes `Student`, `CourseModule`, `SClass` and two association classes `SClassRegistration` and `Enrolment`. The dashed shapes labeled (1) represent the DCSL concern, which captures the structural aspect through four concepts `DClass`, `DAttr`, `DOpt`, `DAssoc` defining entities and associations (e.g., `enrolsIn`, `registersTo`). The dashed shapes labeled (2) denote the AGL concern, which specifies behavioral aspects through `AGraph` defining the reachable state, while the star-like shape '**?**' indicates extensibility for future concerns. These concerns must be systematically composed into the texttt CourseMan problem domain to facilitate the integration and evolution of concern-specific DSLs in a modular and scalable manner.

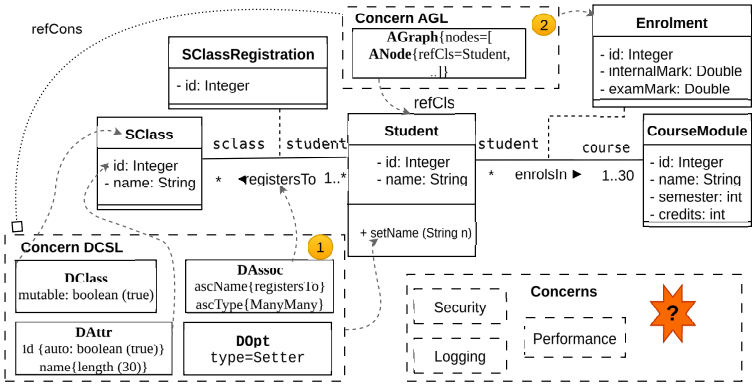

Fig. 1. UML class diagram and concern-specific DSLs in `CourseMan`.

The `CourseMan` system exemplifies a domain characterized by intertwined structural and behavioral concerns. Although concern-specific DSLs can effectively capture their respective aspects, they often lack modularity, expressiveness, and extensibility, which hinders systematic integration and evolution. For example, DCSL [14] supports structural modeling but provides little support for behaviors, roles, constraints, or security, and the addition of further DSLs only amplifies overlaps and inconsistencies. This underscores the need for a unified modeling approach that enables separation of concerns, consistent integration, and continuous evolution of heterogeneous DSLs within a unified executable model. In this context, Fig. 1 illustrates the `refCons` relation, which establishes a cross-reference between the concern-specific DSLs DCSL and AGL.

2.2 Background

To effectively leverage domain expertise, we revisit the foundational concepts that underpin our concern-driven DSL integration approach.

Executable Domain Models in DDD. Domain-Driven Design (DDD) [7] advocates iterative development around a semantically rich domain model that encapsulates core logic and business rules. This model establishes a shared foundation between domain experts and developers through a consistent Ubiquitous Language (UL) [12, 20]. Domain-Specific Languages (DSLs) reinforce this alignment by encoding domain knowledge directly into software artifacts [2, 9], enhancing correctness, maintainability, and extensibility. A major advancement in DDD is the executable domain model, which unifies descriptive and operational semantics [22]. By integrating concerns such as business logic and security into runnable specifications, executable models enhance automation, verification, and adaptability [1]. However, realizing this vision still requires automated composition of heterogeneous concerns—an open challenge in current DDD research.

Composing DSLs for a unified domain model. Two common approaches are used to integrate DSLs: the AST-based method and the meta-modeling method. The AST-based method models languages as hierarchical trees of concepts, where nodes represent elements and edges denote composition or reference relations [23]. It enables direct instance manipulation without parsing, supports multiple syntactic views on a shared structure, and underpins consistency checking, code generation, and traceability. The meta-modeling method, by contrast, defines DSLs at a higher abstraction level using metamodels, providing standardized mechanisms for validation, transformation, and tool interoperability [13]. Our work [15] applies this method by extending UML metaconcepts to integrate concern-specific DSLs into a cohesive metamodel. Each concern is modeled with a dedicated DSL, such as DCSL for structural aspects [14], AGL for behavioral logic [5], while additional DSLs can be incorporated as needed. However, metamodeling approaches face significant limitations: integration processes often rely on complex and fragile mappings, tool support is highly framework-dependent, and extensibility is hindered when the number or heterogeneity of concerns increases. These drawbacks reduce scalability and complicate the handling of crosscutting concerns in practice.

Therefore, this work adopts the AST-based method, leveraging its flexibility, support for direct manipulation, and multi-view integration to compose heterogeneous DSLs into a unified domain model aligned with DDD principles. The goal is to improve extensibility and expressiveness in complex software systems.

3 Overview of Our Approach

This section introduces our approach to composing concern-specific DSLs into a unified domain model. Figure 2 illustrates a four-step iterative process for integrating and refining these DSLs toward automatic software prototype generation.

First, at the language design level (label A), the designer specifies the concern domain using a concern-specific DSL as input. For each concern-specific DSL, the AS, CS, and semantics are formally defined, resulting in a concern-specific DSL that precisely captures the target concern. *Second*, at the application level (label B), the designer utilizes the outputs from the previous step along with

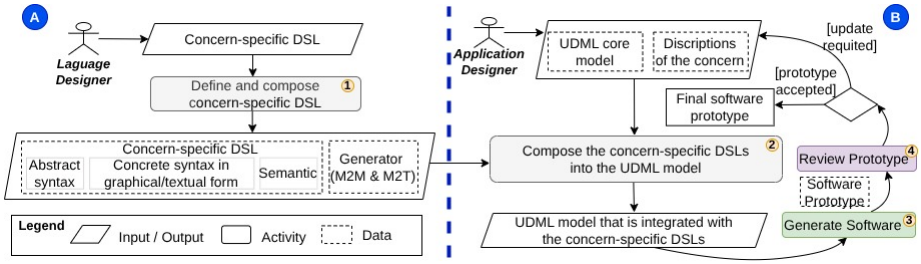

Fig. 2. An overview of our approach, organized into two main layers: (A) the language design level and (B) the language application level.

the existing UDML core model. Along with descriptions of new or changing concern requirements, these inputs help compose the concern-specific DSLs into the UDML model for coherent integration. Textual or graphical CS may be used to construct a detailed model that represents the specific concern instance. The output of this step is a UDML model fully integrated with the given concern-specific DSL. *Third*, the integrated UDML model serves as the foundation for generating production software. The resulting software artifact is then evaluated by the designer to gather feedback. *Finally*, if feedback is provided, both the UDML model and the corresponding concern-specific DSLs are updated accordingly. The process then repeats, supporting a continuous improvement cycle until the software system meets the specified requirements.

4 Composing Concern-Specific DSLs

This section defines the syntax, semantics of UDML and introduces the algorithm for composing heterogeneous DSLs into a unified domain model.

4.1 Defining the Syntax and Semantics of UDML

Our approach to defining the UDML adopts a structured three-layer framework comprising abstract syntax, concrete syntax, and formal semantics, complemented by an integration strategy.

Definition 1. *The **UDML** language is constructed from a core UDML and a set of concern-specific DSLs* $\{DSL_i\}_{i=1}^{n}$*, where each* DSL_i *describes a distinct aspect of the system. Each DSL is specified by a triplet* (AS_i, CS_i, Sem_i)*, where* AS_i*,* CS_i*, and* Sem_i *represent the abstract syntax, the concrete syntax, and the semantics of the **UDML**, respectively.*

Abstract syntax definition (AS): The AS of UDML is formalized as a hierarchical tree structure where each node denotes an instance of a language concept, edges represent composition or reference relations, and attributes capture local properties of nodes. Concern-specific DSLs extend the tree incrementally by introducing new concepts and relations while preserving the common backbone defined in the UDML core.

Definition 2. *Let* C_{UDML} *be the set of meta-concepts of the UDML and* A_{UDML} *the set of attributes applicable to these concepts. For each concept* $c \in C_{UDML}$*, let* $A(c) \subseteq A_{UDML}$ *denote the set of attributes defined for* c*. The **AS** of UDML is defined as the tuple* $AS_{UDML} = (N, root, child, refCons, label, attr, P_{UDML})$*, where:*

- *N: A finite set of nodes.*
- *$root \in N$: The root node representing the unified domain model, instantiated in UDML as* `DomainModel`*.*
- *$child \subseteq N \times N$: A tree relation specifying the ordered children of each node.*
- *$refCons \subseteq N \times N$: A set of cross-reference edges linking nodes across concerns.*
- *$label : N \rightarrow C_{UDML}$: A labeling function assigning each node to a meta-concept.*
- *$attr : N \rightarrow (A(label(n)) \rightarrow V)$: An attribute function assigning values to the attributes of the node's concept.*
- *P_{UDML}: A set of well-formedness rules and constraints on the tree structure.*

Concrete syntax definition (CS): The CS is defined as a projection of the AST into user-facing notations. Each syntactic view provides a domain-appropriate visualization while all views operate on the same underlying AST. This ensures consistency, usability for diverse stakeholders, and synchronization across notations.

Definition 3. *The **CS** of UDML is defined as the tuple $CS_{UDML} = (V, M, \pi, H, P_{CS})$ where:*
- *V: A set of visual symbols (graphical or textual primitives such as boxes, arrows, tables, and tags).*
- *M: A set of mapping rules that associate either AST nodes or edges to visual symbols.*
- *$\pi : N \cup (N \times N) \rightarrow V$: A projection function mapping either AST nodes $n \in N$ or edges (n_i, n_j) to concrete symbols in V.*
- *H: A set of handlers supporting interactive editing actions (e.g., creation, modification, drag-and-drop, and annotation).*
- *P_{CS}: A set of presentation constraints ensuring the validity and consistency of visualizations.*

In addition, the projection function can be indexed by view, i.e., $\pi_v : N \cup (N \times N) \rightarrow V$, where $v \in Views$, allowing multiple concrete notations for the same AST.

Formal semantics specification (Sem): The semantics of UDML is defined by assigning meaning directly to nodes and edges of the abstract syntax tree. Each concern-specific DSL contributes a modular semantic mapping, and these mappings are composed along the AST structure.

Definition 4. *The **Sem** of UDML is defined by a function $\langle Sem \rangle_{UDML} : N \cup (N \times N) \rightarrow D$ where:*
- *N and $N \times N$: the set of AST nodes and edges (child or reference relations).*
- *D: the semantic domain, defined as a product of the concern-specific domains: $D = D_{DSL_1} \times D_{DSL_2} \times \ldots \times D_{DSL_n}$*
- *$\langle Sem \rangle_{UDML}$: the semantics of a node $n \in N$, determined by the rules of its corresponding concern-specific DSL.*
- *$\langle (n_i, n_j) \rangle_{UDML}$: the semantics of an edge, defined by how parent-child or cross-reference relations propagate semantics.*

The global semantics $\langle Sem \rangle_{UDML}$ is defined as the composition of the local semantics of all nodes and edges in the unified AST. Each constituent DSL retains its own interpretation rules within the unified structure, thereby preserving its original semantics while ensuring consistency across concerns. The unification of concern-specific DSLs into UDML is performed incrementally through the tree-merging Algorithm 1, where concepts are combined via inheritance and cross-cutting concerns are attached using annotations. A complete formal specification is provided in the long version of this paper [16].

Algorithm 1 Tree-merging algorithm to integrate concern-specific DSLs into UDML

Input: $D = \{DSL_1, DSL_2, \ldots, DSL_n\}$: A set of concern-specific DSLs, each specified as a triplet (AS_i, CS_i, Sem_i)

Output: $UDML_{unified}$: The unified UDML comprising a global AS, CS, and Sem

1: Initialize: $AS_{UDML} \leftarrow AS_{core}$, $CS_{UDML} \leftarrow CS_{core}$, $Sem_{UDML} \leftarrow Sem_{core}$
2: **for** each $DSL_i = (AS_i, CS_i, Sem_i) \in D$ **do** ▷ //Merge Abstract Syntax
3: **for** each concept $c \in AS_i$ where $c \notin AS_{UDML}$ **do**
4: **if** c extends core concept **then**
5: Integrate inheritance into AS_{UDML}
6: **end if**
7: Add attributes, relations, and constraints of c to AS_{UDML}
8: **end for** ▷ //Merge Edges (child, ref)
9: **for** each edge $(n_i, n_j) \in child_i \cup refCons_i$ **do**
10: **if** $(n_i, n_j) \notin (child_{UDML} \cup refCons_{UDML})$ **then**
11: Add edge (n_i, n_j) into AS_{UDML}
12: **else**
13: Resolve conflicts by applying mapping or priority rules
14: **end if**
15: **end for** ▷ //Merge Concrete Syntax
16: **for** each new or extended concept c **do**
17: Add graphical or textual symbols for c into CS_{UDML}
18: Extend UI handlers (e.g., drag-drop, annotations, editing operations)
19: **end for** ▷ //Merge Semantics
20: Integrate Sem_i into Sem_{UDML} modularly
21: Compose new constraints into P_{UDML}
22: Check cross-concern consistency between Sem_i and Sem_{UDML}
23: **end for**
24: **return** $UDML_{unified} = (AS_{UDML}, CS_{UDML}, Sem_{UDML})$

4.2 A Mechanism for Composing concern-specific DSLs in UDML

A unified AST is constructed from multiple concern-specific DSLs, centered on the UDML Core and its extensions (e.g., DCSL, AGL). Each node represents an instance of a metaconcept (e.g., `Student:DClass`), and edges capture composition or reference relations (e.g., `AGraph` $\rightarrow$ `DClass`). Algorithm 1 outlines the incremental merging of concern-specific DSLs into the UDML framework. The algorithm initializes the UDML with a core module that serves as the base for all extensions (line 1), then iteratively merges each input DSL (line 2). It first integrates the AS: for each concept in the current DSL, the concept is added to

the UDML's AS if not already present (lines 3–15). Next, the CS is merged (lines 16–19), followed by modular integration of formal semantics, such as mapping AGL semantics to a state machine (lines 20–22). After all DSLs are processed, the algorithm outputs the unified UDML (line 24), resulting in an executable model that preserves modularity and traceability across diverse concerns.

5 Tool Support and Experiments

This section focuses on presenting the support tool we developed for our proposed method, called `UDML/MPS`. We also evaluate our method, showing that it outperforms other approaches.

The `UDML/MPS` tool is implemented in JetBrains MPS [3, 21], which offers a flexible AST-based platform to define and compose DSLs. Each concern-specific DSL defines reusable AST constructs that represent the core concepts of the concern, while its projection editor provides a dedicated interface and simultaneously contributes to a shared model instance. We have implemented `UDML/MPS` as an open-source project on GitHub[3]. Specifically, Fig. 3 provides an overview of the `UDML/MPS` tool. On the left, label (1) shows the JetBrains MPS projectional editing environment for the `UDML/MPS` tool, which provides mechanisms to define the core UDML and its concern-specific DSLs, as well as to compose them. This tool consists of two main components: First, the `Language` component defines the AS, CS, and semantics of DSLs, including the core UDML and its concern-specific DSLs, via the `Structure`, `Editor`, `Constraints`, `Behavior`, and `Generator` aspects for generating Java code. Second, the `Solution` component defines UDML models—including concern-specific DSLs and code generated by the MPS generator—while the `Sandbox` provides a testing environment for developing and validating UDML models.

On the right, label (2) shows the graphical interface for applying the `UDML/MPS` tool to a concrete UDML model—in this case, `CourseMan`. To create a unified domain model for `CourseMan`, we first define a core model within the `Sandbox`. We then incorporate concerns in the corresponding DSLs using a drag-and-drop interface. This environment enables seamless model composition through classes, relationships, and annotations according to the specified requirements, followed by automatic code generation for the `JDA` framework. A more detailed description is provided in the extended version of this paper [16].

Experimental results. We have applied our method to the motivating case study, `CourseMan`, as well as to another case study, `OrderMan`. Due to limited space, a detailed explanation of the experiments is only provided in the long version of this paper [16]. These case studies highlight the practical applicability of our method and emphasize the two main benefits it provides. First, the `UDML/MPS` tool allows us to define mapping rules and templates that transform structured models into executable domain models embedded in object-oriented code (e.g., Java). By unifying model definition and code generation within a single language engineering environment, it inherently ensures model–code consistency. Second, the `UDML/MPS` tool generates executable domain models, reduc-

[3] https://github.com/vinhskv/udml-syntax-soict2025

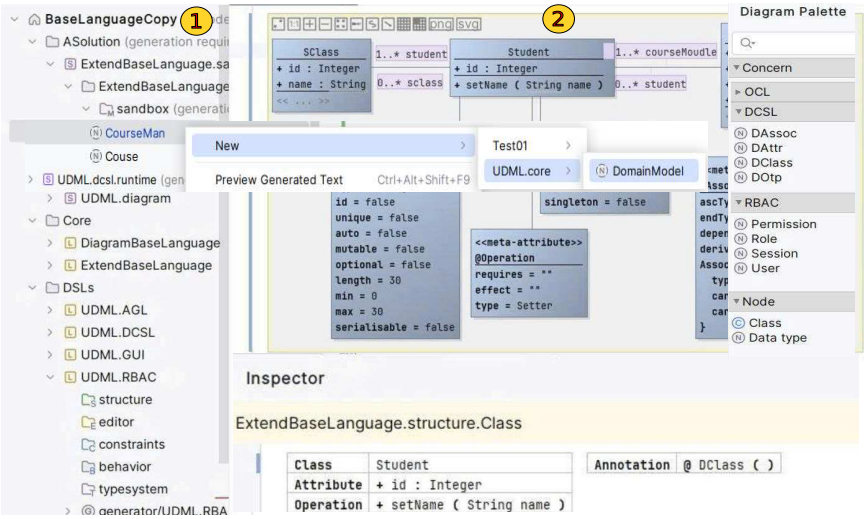

Fig. 3. MPS-Based implementation of concern-specific DSL integration in UDML.

ing semantic drift, simplifying DSL evolution, and maintaining synchronization between models and implementations, while its type-safe transformations and fine-grained control over intermediate models further improve maintainability, refactoring, and system scalability.

Discussion. We have successfully applied `UDML/MPS` to several problem domains in practice. However, our method raises several potential validity concerns, as outlined below. First, the correctness of our method relies on the expressive adequacy of individual concern-specific DSLs and their coherent integration through UDML, consistent with DDD principles. Second, integrating heterogeneous external DSLs for the corresponding concerns requires adequate syntactic support, currently ensured through systematic metaconcept validation, interface design, transformation testing, and generator rule verification. Last but least, the generated Java code may misalign with requirements if generation errors occur within the JDA framework. To mitigate this, we conduct careful review and verification of the generated output.

6 Related Work

We position our work at the intersection between the following areas: DSL engineering, DDD concern-specific DSLs, Integrating concern-specific DSLs.

DDD and Concern-Specific DSLs. In the DDD paradigm [20], the domain model and ubiquitous language are central. However, as systems increase in scale and complexity, they require modeling and integrating multiple concerns—such as business logic, security, UI, and performance—often realized as separate DSLs [1]. Prior studies show that addressing these concerns in isolation leads to fragmentation, inconsistencies, and reduced traceability between the domain model and implementation.

Integrating Concern-Specific DSLs. DSL engineering has matured significantly [22], with language workbenches such as JetBrains MPS [3] enabling the design, extension, and composition of DSLs for both academic and industrial use. MPS has been applied across domains including finance, healthcare, safety-critical systems, and industrial automation, where tailored DSLs improve productivity, quality, and domain expertise utilization. Recent research has emphasized systematic methods for integrating heterogeneous concern-specific DSLs [11, 17]. Platforms like JetBrains MPS [3] have advanced this field by enabling modular language development through reuse and composition, thereby enhancing efficiency and quality.

Our earlier work [15] proposed composing concerns into an executable unified domain model using a UML-based metamodel. Concerns were specified in external DSLs and mapped to internal DSLs to produce executable models embedded in object-oriented languages. In contrast, this work defines the full syntax of concern-specific DSLs and introduces an AST-level integration method based on tree merging, along with a unified syntax for their composition.

Nevertheless, existing approaches often fall short in supporting the complete DSL development pipeline—spanning editing, verification, code generation, and automated testing. Backward traceability between models and generated code remains a critical challenge, particularly as systems evolve.

7 Conclusion and Future Work

This paper introduces a methodology for composing heterogeneous concern-specific DSLs into a unified domain model and defines the complete syntax and semantics of the UDML to support domain-driven development through an annotation-based composition mechanism at the AST level. UDML structures domain concerns as an AST, where each node represents a DSL concept, providing a unified yet modular foundation that ensures consistency across structural, behavioral, and security aspects. It further enables direct manipulation and multi-view editing, bridging domain modeling with executable implementation. The approach was prototyped in JetBrains MPS and integrated with the JDA framework for code generation.

In the future, we plan to develop an MPS plug-in for our method for many domain concerns. We also intend to develop a technique for automatically transforming a specification requirement into complex software systems.

References

1. Andrzej Wąsowski, Thorsten Berger: Domain-Specific Languages Effective Modeling, Automation, and Reuse. Springer Cham (2023)
2. Brambilla, M., Cabot, J., Wimmer, M.: Model-Driven Software Engineering in Practice. Morgan&Claypool, 2nd edn. (2017)
3. Bucchiarone, A., Cicchetti, A., Ciccozzi, F., Pierantonio, A.: Domain-specific languages in practice: with JetBrains MPS. Springer Nature (2021)
4. Buede, D.M., Miller, W.D.: The Engineering Design of Systems: Models and Methods. Wiley, 4th edn. (2024)

5. Dang, D.H., Le, D.M., Le, V.V.: AGL: Incorporating behavioral aspects into domain-driven design. Information and Soft.Tech **163**, 107284 (2023)
6. D.M. Le, D.-H. Dang, V.-H. Nguyen: Generative software module development for domain-driven design with annotation-based domain specific language. Information and Software Technology **120**, 106–239 (2022)
7. Evans, E.: Domain-Driven Design: Tackling Complexity in the Heart of Software. Addison-Wesley Professional (2004)
8. Foster, E., Jr, B.T.: Software Engineering: A Methodical Approach, 2nd Edition. Auerbach Publications, New York, 2 edn. (Jul 2021)
9. Fowler, M.: Domain-Specific Languages. Addison-Wesley Professional, 1st edn. (Sep 2010)
10. France, R., Rumpe, B.: Model-driven development of complex software: A research roadmap. In: Future of Software Engineering (FOSE'07). pp. 37–54. IEEE (2007)
11. Haber, A., Look, M., Perez, A.N., Nazari, P.M.S., Rumpe, B., Völkel, S., Wortmann, A.: Integration of heterogeneous modeling languages via extensible and composable language components. In: 3rd Int. Conf. on Model-Driven Engineering and Software Development (MODELSWARD). pp. 19–31. IEEE (2015)
12. Jaiswal, S.K., Agrawal, R.: Domain-Driven Design (DDD)- Bridging the Gap between Business Requirements and Object-Oriented Modeling. Int. Innovative Research in Engineering and Management **11**(2), 79–83 (2024), number: 2
13. Katara, M., Katz, S.: A concern architecture view for aspect-oriented software design. Software & Systems Modeling **6**(3), 247–265 (Aug 2007)
14. Le, D.M., Dang, D.H., Nguyen, V.H.: On domain driven design using annotation-based domain specific language. Computer Languages, Systems & Structures **54**, 199–235 (Dec 2018)
15. Le, V.V., Dang, D.H.: An Approach to Composing Concerns for an Executable Unified Domain Model. In: Int. Conf. On Computing and Communication Technologies (RIVF). pp. 424–428 (Dec 2024), iSSN: 2473-0130
16. Le, V.V., Nguyen, N.H., Nguyen, D.Q., Duc-Hanh: A method for composing concerns into a unified domain model in domain-driven design. Tech.Rep., VNU (2025), https://byvn.net/lyCO
17. Nosál', M., Sulír, M., Juhár, J.: Language composition using source code annotations. Computer Science and Information Systems **13**(3), 707–729 (2016)
18. Ouchani, S., Debbabi, M.: Specification, verification, and quantification of security in model-based systems. Computing **97**(7), 691–711 (Jul 2015)
19. Suryn, W.: Software Quality Engineering: A Practitioner's Approach. John Wiley & Sons (Dec 2013), google-Books-ID: 1dNiAgAAQBAJ
20. Vernon, V.: Implementing domain-driven design. Addison-Wesley Professional, 1 er edn. (2013)
21. Voelter, M.: Language and IDE Modularization and Composition with MPS. Lecture Notes in Computer Science (LNPSE), vol. 7680, pp. 383–430. Springer, Berlin, Heidelberg (2013)
22. Völter, M., Benz, S., Dietrich, C., Engelmann, B., Helander, M., Kats, L., Visser, E., Wachsmuth, G.: DSL Engineering - Designing, implementing and using domain-specific languages. M Volter / DSLBook.org, Stuttgart, Germany (2013)
23. Würthinger, T., Wöß, A., Stadler, L., Duboscq, G., Simon, D., Wimmer, C.: Self-optimizing AST interpreters. Proc. 8th symposium on Dynamic languages pp. 73–82 (Oct 2012). https://doi.org/10.1145/2384577.2384587